

```
=====
MINIIDE.IMG – TUDO QUE SABEMOS SOBRE A IMAGEM DA MiniIDE
(CoCo DSK & CCC Utility – anotações de engenharia, 2026-06)
Fonte: análise byte-a-byte de MiniIDE_Backup_v3.img e MiniIDE_Backup_v4.img
(originais em G:\...\Interfaces_MiniIdeCoCoSDCDriveWireCartDSK\MiniIDE\)
```

0. O QUE É

A MiniIDE é uma interface IDE/CompactFlash para o Tandy Color Computer (CoCo), criada por Victor Trucco. O arquivo .img é o dump BRUTO de um cartão CF e contém DOIS MUNDOS

independentes lado a lado no mesmo cartão:

- (1) uma PARTIÇÃO OS-9 (sistema NitroS-9 bootável), e
- (2) a região RS-DOS/Disk BASIC do HDB-DOS, com 256 "discos" virtuais navegados pelo browser SIDEKICK.

Tamanho do dump analisado: 507.379.712 bytes ($\approx$ 507 MB) – v3 e v4 têm o mesmo tamanho.

1. LAYOUT FÍSICO (offsets confirmados na v3)

```
-----
[0 ————— 134.217.728)  PARTIÇÃO OS-9 (RAW)                = 128,0 MB
[134.217.728 — 173.783.040)  FLASH APAGADO / FOLGA (0xFF)       $\approx$  37,7 MB
[173.783.040 — 256.358.400)  REGIÃO RS-DOS/SIDEKICK (256 slots)  $\approx$  78,8 MB
[256.358.400 — 507.379.712)  Resto do cartão (não usado/n/ característico)  $\approx$  240
MB
```

- O início do cartão é RESERVADO ao OS-9 (a partição) + uma folga.
 - O "offset do disco 000" (173.783.040 = 165,73 MB) é a fronteira p/ a região RS-DOS.
- No mundo RGBDOS/HDBDOS isso é uma VARIÁVEL DE FIRMWARE (o "RGB sector offset"), lida da ROM (no RGBDOS clássico: PEEK &HD938..&HD93A; ver hrsdos.c / SPECS.BAS).
- Nosso app NÃO lê essa variável: DETECTA a âncora dinamicamente (1ª sequência de 3 discos RS-DOS dobrados consecutivos) → funciona mesmo se o offset variar entre imagens.

2. A PARTIÇÃO OS-9 (offset 0)

Formato: OS-9 / NitroS-9 RBF (Random Block File), setor de 256 B, endereçado por LSN.

Volume (DD.NAM): "NitroS-9 6309 Level 2 v2.3.9"

LSN0 (Setor de Identificação) medido:

DD.TOT = 524.288 setores → $524288 \times 256 = 134.217.728$ B = 128 MB
DD.TKS = 32 (setores/trilha) DD.BIT = 2 (setores por cluster)
DD.MAP = 32.768 bytes (bitmap de alocação, no LSN1)
DD.DIR = LSN 129 (FD do diretório-raiz) DD.FMT = ... DD.SPT = 32

Conteúdo: 1.662 arquivos, 154 diretórios, $\sim$ 105,8 MB livres (maxDepth 5).

Raiz: CMDS SYS DEFS sysgo startup NITROS9 EXTRAS APPS GAMES MUSIC FILTERS
TMP

É BOOTÁVEL: tem OS9Boot (bootfile/kernel) + sysgo + startup + CMDS/SYS/DEFS –
quando o

CoCo entra em OS-9 pela MiniIDE, dá boot a partir desta partição.

IMPORTANTE: a partição OS-9 é RAW (setor de 256 B nativo) – NÃO é sector-doubled.

Conteúdo: ~1.576 módulos OS-9 (extração validada por CRC de módulo = 0x800FE3, a
constante do OS9Defs). Tipos: Program/DevDesc/Driver/FileMgr/System/Subr/Data;
linguagens 6809 / Basic09 / Pascal / C.

Acesso no app: botão "OS-9" no contêiner → navegador hierárquico (somente-leitura).

É UM SÓ sistema, não vários "discos" – não entra na régua 000–255.

3. A FOLGA (128 MB → 165,7 MB)

≈ 37,7 MB de flash APAGADO (todo 0xFF). Não é partição, não é navegável, sem
conteúdo.

Não vira "mais discos" sem reconfigurar o firmware (e mesmo assim 256 é teto). No
máximo

serviria para EXPANDIR a partição OS-9 (mas isso é OS-9, não discos RS-DOS).

4. A REGIÃO RS-DOS / HDB-DOS / SIDEKICK (256 discos)

-
- Âncora (disco 000): offset 173.783.040 (165,73 MB).
 - HDB-DOS endereça os drives 000..255 a partir daí; 256 é TETO RÍGIDO (nº de drive
= 1 byte).
 - Cada slot = 322.560 bytes (um disco RS-DOS de 35 trilhas, SECTOR-DOUBLED).
256 × 322.560 = 82.575.360 B → termina em 256.358.400 (244,5 MB). Tudo cabe no
cartão.
 - Ocupação medida na v3: 237 ocupados + 12 vazios + 7 não-RS-DOS = 256.
vazios (FAT-ish válida, 0 arquivos): ex.: 80, 123, 170, 224, 231, 250, 252,
253, 254, 255
não-RS-DOS (FAT inválida / sobra): ex.: 122, 158, 160, 162, 208, 209, 251
(251 tem SOBRAS de texto OS-9: "system", "Level I", "Password", "Process" – lixo
num slot RS-DOS.)
 - O maior slot OCUPADO é 249 → por isso, antes da "Fase 1", a régua parava em 249.
Agora a
régua cobre 000–255 inteiros (uma entry por slot físico:
ocupado/vazio/não-RS-DOS).

5. SECTOR-DOUBLING (HDBDOS)

Cada setor LÓGICO de 256 B é gravado DUAS VEZES seguidas (S0 S0 S1 S1 ...), então um
disco

RS-DOS de 161.280 B ocupa 322.560 B físicos no slot.

- deDoubleDisk: pega 1 setor de cada par (a metade PAR) → recupera os 161.280 B
padrão.

- reDoubleDisk: escreve cada setor duas vezes.
- ACHADO: ao MODIFICAR um disco existente, as metades ÍMPARES NÃO são cópias idênticas (sobras do CF). Por isso a escrita in-place (image-write-slot) sobrescreve só os chunks PARES de 256 B e PRESERVA os ímpares byte-a-byte (alteração mínima e reversível).
- O doubling é EXCLUSIVO desta região RS-DOS. A partição OS-9 (offset 0) é RAW.

Offsets úteis DENTRO do slot doubled (322.560 B):

FAT (de-doubled trilha17 setor2) → doubled 614×256 = 157.184

DIR (de-doubled trilha17 setor3) → doubled 616×256 = 157.696

NOME SIDEKICK (de-doubled LSN322) → doubled 322×512 = 164.864

Offsets no disco DE-DOUBLED (161.280 B):

FAT = LSN 307 = 78.592 DIR = LSN 308..316 = 78.848..81.152 NOME = LSN 322 = 82.432

6. FORMATO DO DISCO RS-DOS (dentro de cada slot, já de-doubled)

-
- 35 trilhas × 18 setores × 256 B = 161.280 B.
 - Trilha 17 = trilha de diretório (não entra na alocação de granules).
 - setor 2 (LSN 307) = FAT/granule map (68 granules)
 - setores 3-11 (LSN 308-316) = diretório (72 entradas de 32 B)
 - setores 12-18 = livres no RS-DOS padrão (o SIDEKICK usa o setor ~17/LSN322 p/ o nome)
 - 68 granules (2 por trilha × 34 trilhas não-diretório), 2.304 B cada.
 - FAT (cada byte): 0xFF = livre; < 68 = próximo granule da cadeia; 0xC0-0xC9 = fim de arquivo (1-9 setores no último granule).
 - Entrada de diretório (32 B): nome(8) ext(3) tipo(1) flagASCII(1) 1ºGranule(1) bytesNoÚltimoSetor(2) + reservado. tipo: 0=BASIC,1=Data,2=ML,3=Source. flag: 0=bin,0xFF=ASCII.
 - 1º byte 0x00 = entrada vazia/apagada; 0xFF = fim.

7. NOMES DE DRIVE DO SIDEKICK

-
- O SIDEKICK ("R=RENAME DRIVE") guarda um NOME para cada drive em LSN 322 (de-doubled), offset +164.864 no slot doubled. 8 bytes ASCII terminados em NUL. Confirmado em v3 e v4 (mesmo offset). É SEGURO ler: a trilha 17 nunca guarda dados de arquivo.
- Cobertura: só drives que o usuário RENOMEOU têm nome (v3: 49/237; v4: 37). O resto fica em branco → cai no fallback (nomes de arquivo).
 - O setor 322 é, na verdade, "nome(8B) + NUL + um cache de diretório" do SIDEKICK (estudar a fundo antes de ESCREVER nomes em drives sem nome – Fase B do roadmap MiniIDE).

- Exemplos reais (v3): 000=MINIIDE, 001..=GAMES01.., 060..070=DRAGON01..DRAGON11, 071=CATACOMB, 072=CHATWIG, 074=CHESS, 075=CITY BOM(BER), 077=CC3 CLUE, 078=COCOZONE, 079=SOLITAIR.
 - "DRAGON01..11" são NOMES DE DRIVE, NÃO discos Dragon. O conteúdo é RS-DOS (jogos CoCo).
- Detecção de Dragon-format real na imagem inteira = 0.

8. DISCOS DE "ARTE NO DIR" (caracteres semigráficos do VDG 6847)

Alguns discos desenhavam figuras no comando DIR usando bytes SEMIGRÁFICOS/controlados nos NOMES de arquivo (truque de 8-bits, estilo PETSCII do C64). Exemplos: 072 (CHATWIG) e 075 (CITY BOMBER). São discos RS-DOS VÁLIDOS (FAT ok) com arquivos reais + entradas decorativas (muitas apontam para o granule 0 – só efeito visual).

- O parser ANTIGO descartava/corrompia esses nomes (usava 'ascii' que mascara o bit 7, e um filtro hasPrintable). Isso (a) ESCONDIA arquivos (ex.: um arquivo de nome [8D]x8 = semigráfico-4 entre P7.BIN e INFO.BAS no disco 072), e (b) marcava o disco inteiro como "não suportado" (ex.: 075, 53% dos nomes "ilegíveis").

Pior: qualquer reescrita de diretório (ordenar) PERDIA esses arquivos → granules órfãos → corrupção no hardware.

- CORRIGIDO (Fase 0): parseDsk preserva os bytes EXATOS (rawName/rawExt), exibe com os bytes gráficos, NÃO descarta nada; sortDskDirectory copia os 32 bytes verbatim.

Os discos de arte agora LISTAM e EXTRAEM, marcados "🌀 arte · 📖 leitura", com EDIÇÃO BLOQUEADA (para não embaralhar o desenho). O █ é só TELA – o hexadecimal gravado é o real, byte-a-byte.

9. DETECÇÃO (como o app decide o formato)

Ordem do discriminador (regra OBRIGATÓRIA descoberta com discos em branco): um disco OS-9

TAMBÉM passa em isRsDosDisk (ambiguidade), mas RS-DOS nunca passa em OS-9. Logo: OS-9(estrito) → Dragon → RS-DOS → desconhecido

- isOs9DiskStrict: base + cluster potência-de-2 + raiz começa com "."/".." apontando p/ o próprio FD; confiável no OFFSET 0 (não varrer byte a byte – flash apagado dá ~3 falsos/100 MB).
- Na abertura da imagem: FAT(CoCoSDC) → DriveWire → scan MiniIDE(âncora) → OS-9@0 → dsk.
- read-dsk-directory: checa OS-9 antes de RS-DOS; devolve rsdos (FAT válida?) e

hasArt.

"Não suportado" = só quando NÃO é RS-DOS (OS-9/dupla-face/JVC). Art = RS-DOS válido (lista).

10. REFERÊNCIAS DE FORMATO (estudo; sem copiar GPL)

-
- hrsdos.c + SPECS.BAS (Robert Gault): ponte RS-DOS↔OS-9 num HD RGBDOS; documentam o "base = RGB offset" e o modelo "OS-9 + RS-DOS separados por offset".
 - dosdir.asm (Todd Wallace, licença permissiva) + OS9Defs: formato RBF do OS-9.
 - NitroOS-9 / Color Computer Toolshed: referência (re-implementado limpo em TS).

=====

FIM – manter atualizado conforme novas descobertas (v4, RetroRewind, hardware).

=====